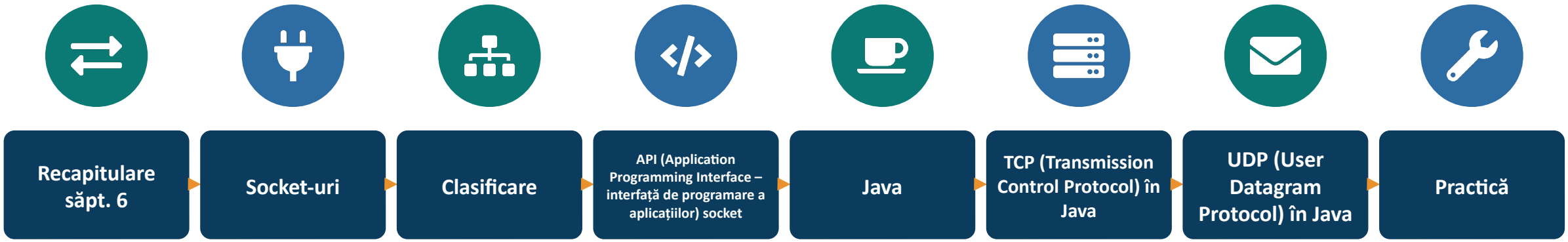
Comunicația prin socket-uri

Săptămâna 7 — Scheme pentru învățare: definiții, clasificare, API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket, programare client-server TCP (Transmission Control Protocol) și UDP (User Datagram Protocol) în Java

Săptămâna 7 pe scurt: de la nivelul transport la programe funcționale



Recapitulare săpt. 6	Socket-uri	Clasificare	API (Application Programming Interface – interfață de programare a aplicațiilor) socket	Java	TCP (Transmission Control Protocol) în Java	UDP (User Datagram Protocol) în Java	Practică
TCP (Transmission Control Protocol), UDP (User Datagram Protocol), porturi, 5-tuplu	definiții	familie, tip, rol, I/O (Input/Output – intrare/ieșire), livrare	secvențe de apeluri	clasele java.net	ServerSocket, Socket	DatagramSocket, Packet	comparare, depanare, testare

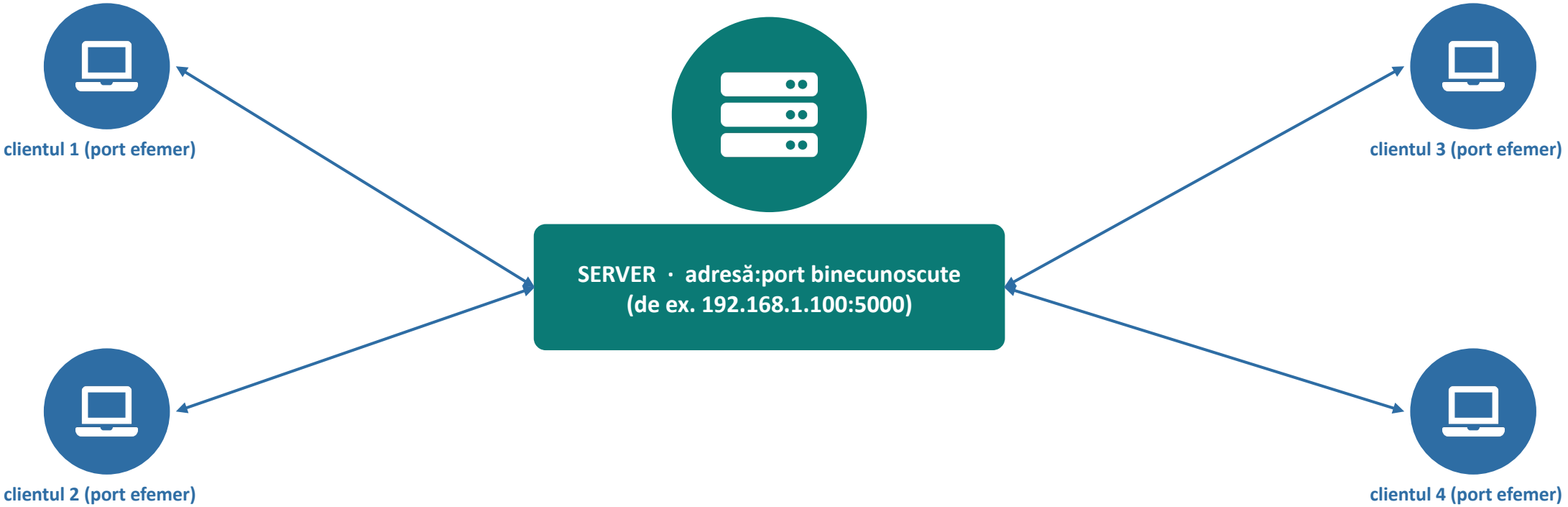
Ideea-cheie a săptămânii: aplicațiile nu construiesc niciodată singure segmente TCP (Transmission Control Protocol) sau datagrame UDP (User Datagram Protocol). Ele cer sistemului de operare un SOCKET și citesc/scriu date prin el; nucleul rulează dedesubt TCP (Transmission Control Protocol)/UDP (User Datagram Protocol)/IP (Internet Protocol). Java încapsulează API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket în câteva clase din pachetul java.net.

1

De la serviciile de transport la aplicații

Modelul client-server și locul socket-ului

Modelul client-server



Server

- pasiv: așteaptă cereri
- mereu pornit, port fix
- servește mulți clienți

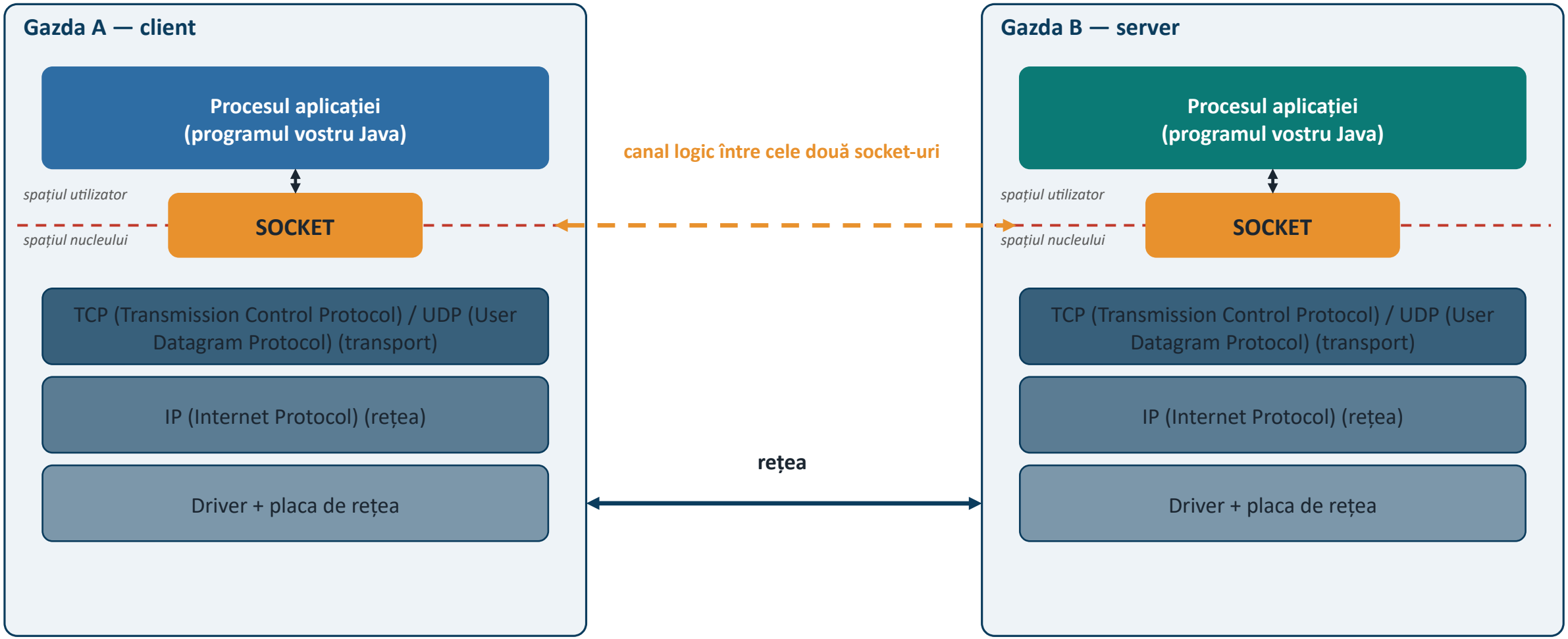
Client

- activ: inițiază contactul
- rulează când e nevoie
- OS (Operating System – sistem de operare) îi alege portul

Peer-to-peer (contrast)

- fiecare nod este client ȘI server
- același API (Application Programming Interface – interfață de programare a aplicațiilor) socket dedesubt

Unde se află socket-ul: ușa dintre aplicație și nucleu



Tot ce este deasupra socket-ului ține de programator; tot ce este dedesubt face OS (Operating System – sistem de operare).

2

Socket-uri: definiții

Capăt de comunicație, adresă, handle, conexiune

Ce este un socket? Un capăt de comunicație, trei fețe

SOCKET
capăt al unui canal bidirecțional

Pentru aplicație → un HANDLE

- Linux/Unix: descriptor de fișier (int)
- Windows: handle SOCKET
- Java: un obiect Socket / DatagramSocket

Pentru rețea → o ADRESĂ DE SOCKET

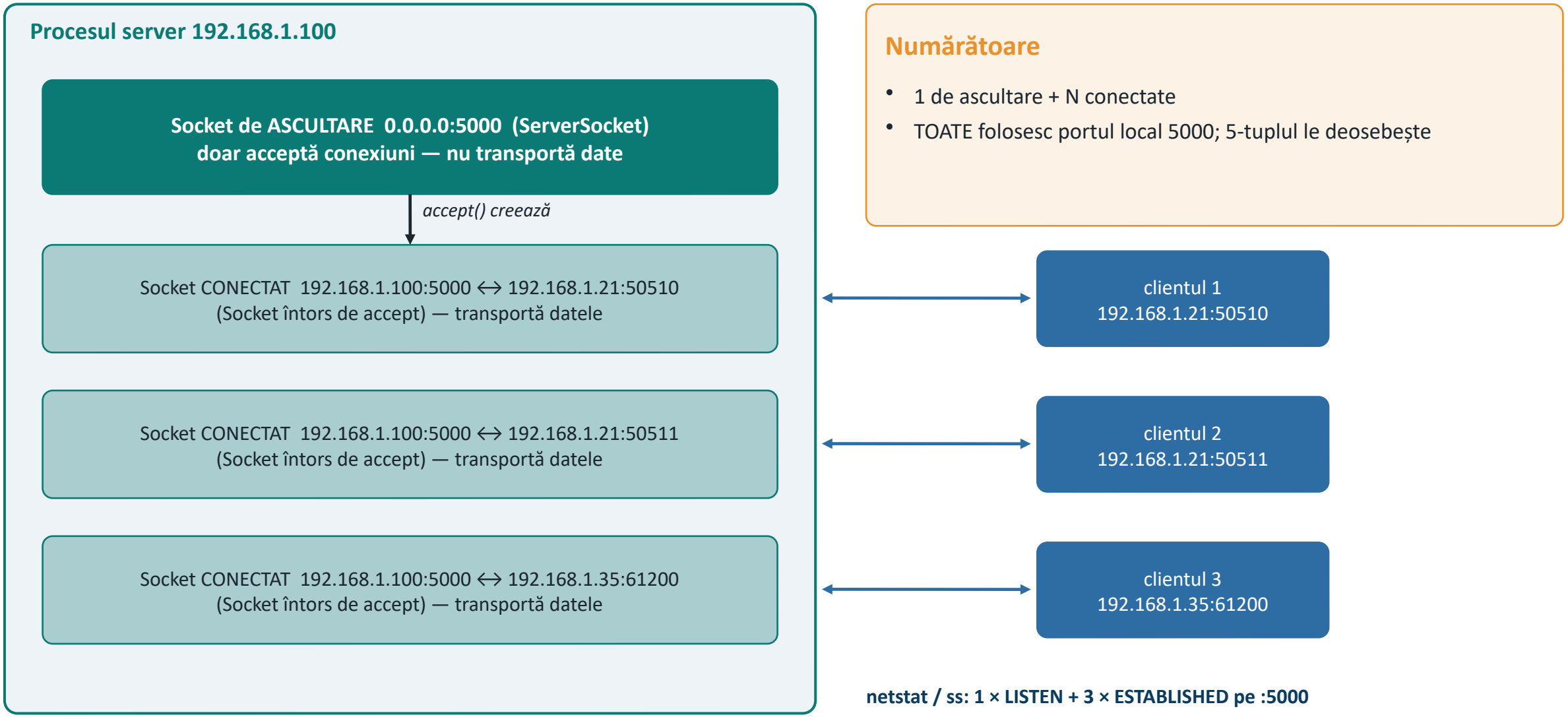
- (adresă IP (Internet Protocol), port)
- e.g. 192.168.1.10:5000
- Java: InetSocketAddress

Pentru OS (Operating System – sistem de operare) → un PROTOCOL + stare

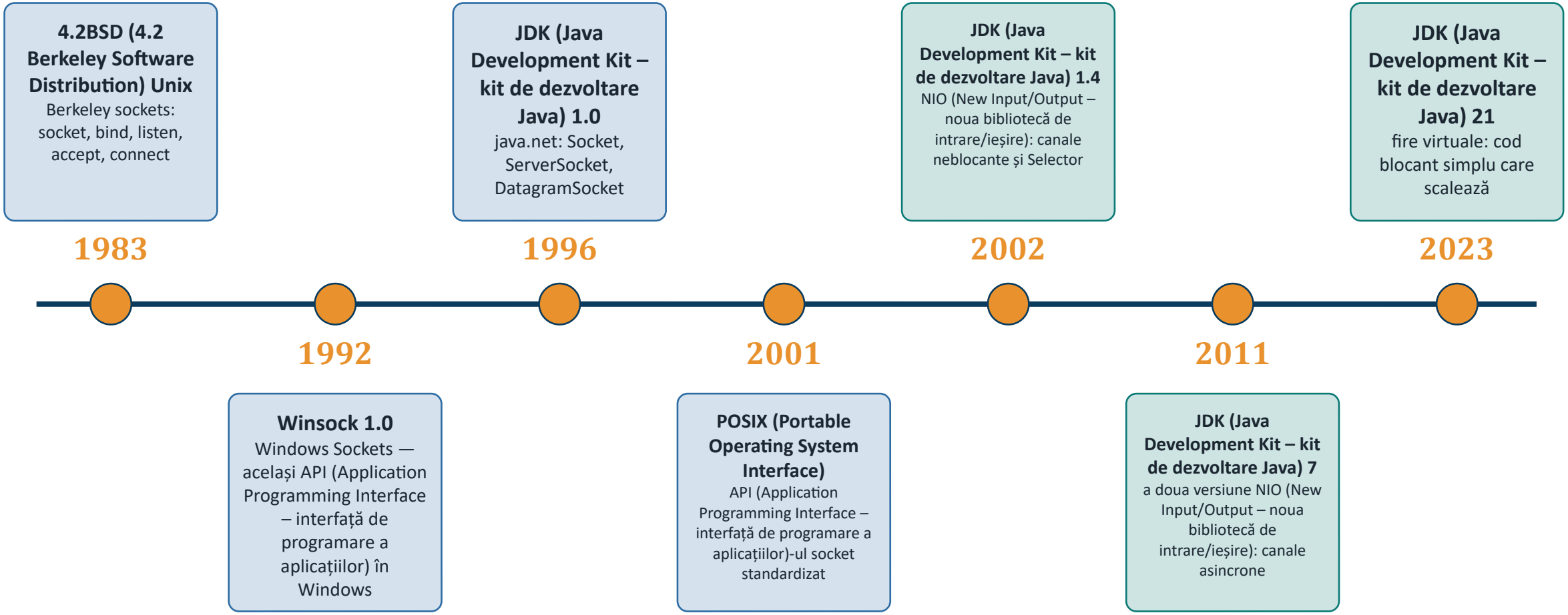
- TCP (Transmission Control Protocol) sau UDP (User Datagram Protocol)
- buffere, temporizatoare, stare (LISTEN, ESTABLISHED…)
- se află în nucleu

Conexiune (TCP (Transmission Control Protocol)) = pereche de socket-uri = 5-tuplu (protocol, IP (Internet Protocol) local, port local, IP (Internet Protocol) la distanță, port la distanță) — legătura cu săptămâna 6.

Socket de ascultare comparativ cu socket-uri conectate (server TCP (Transmission Control Protocol))



Scurt istoric al API (Application Programming Interface – interfață de programare a aplicațiilor)-ului socket



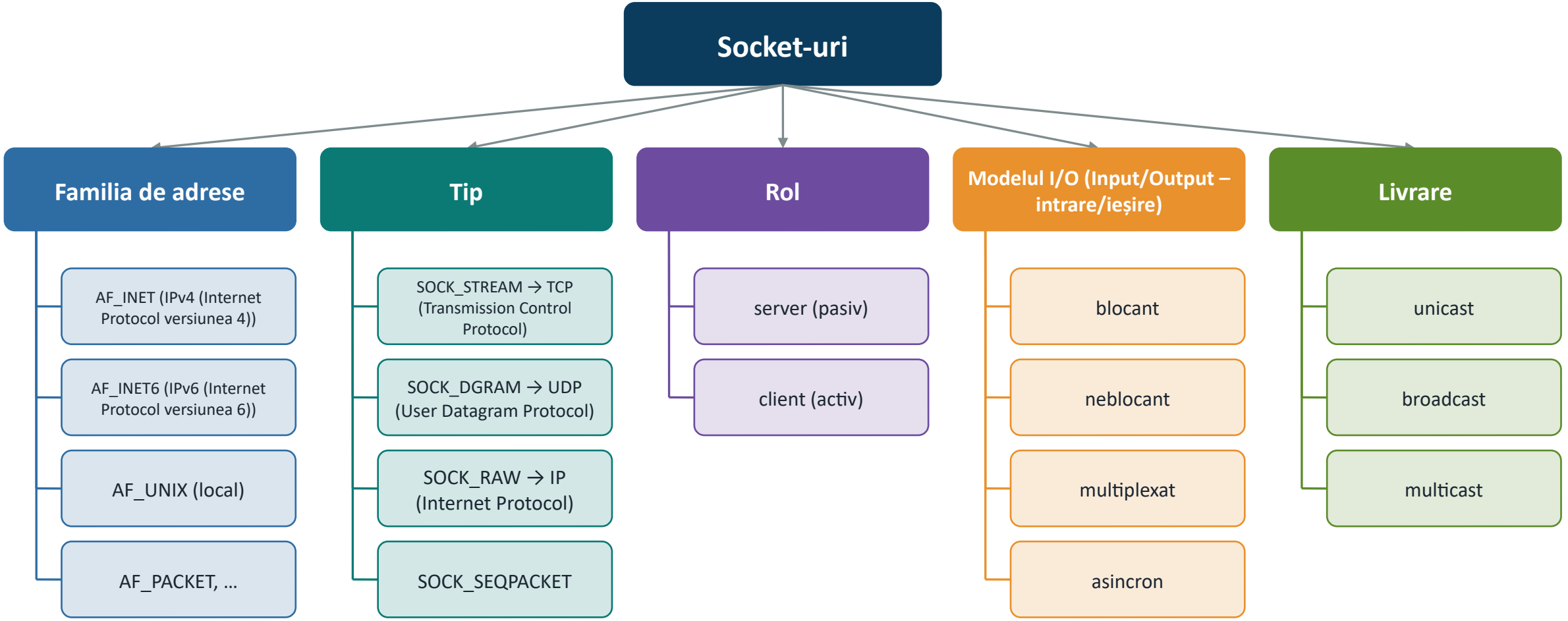
Aceleași primitive de 40 de ani: ce învățați cu Java se transpune direct în C, Python, C#, Go, Rust…

3

Clasificarea socket-urilor

Cinci criterii independente

Arborele de clasificare: cinci criterii



În acest curs: socket-uri Internet (AF_INET/AF_INET6), tipurile stream și datagramă, I/O (Input/Output – intrare/ieșire) blocant, unicast.

După tip: socket-uri stream, datagramă și raw

SOCK_STREAM (TCP (Transmission Control Protocol))

H E L L O W O R L D …

o conductă continuă de octeți

- orientat pe conexiune
- fiabil, ordonat
- fără limite ale mesajelor
- Java: Socket, ServerSocket
- web, e-mail, SSH (Secure Shell), fișiere

SOCK_DGRAM (UDP (User Datagram Protocol))



- fără conexiune
- mesaje independente
- limitele sunt păstrate
- se pot pierde / reordona
- Java: DatagramSocket, DatagramPacket
- DNS (Domain Name System – sistemul numelor de domeniu), DHCP (Dynamic Host Configuration Protocol), streaming

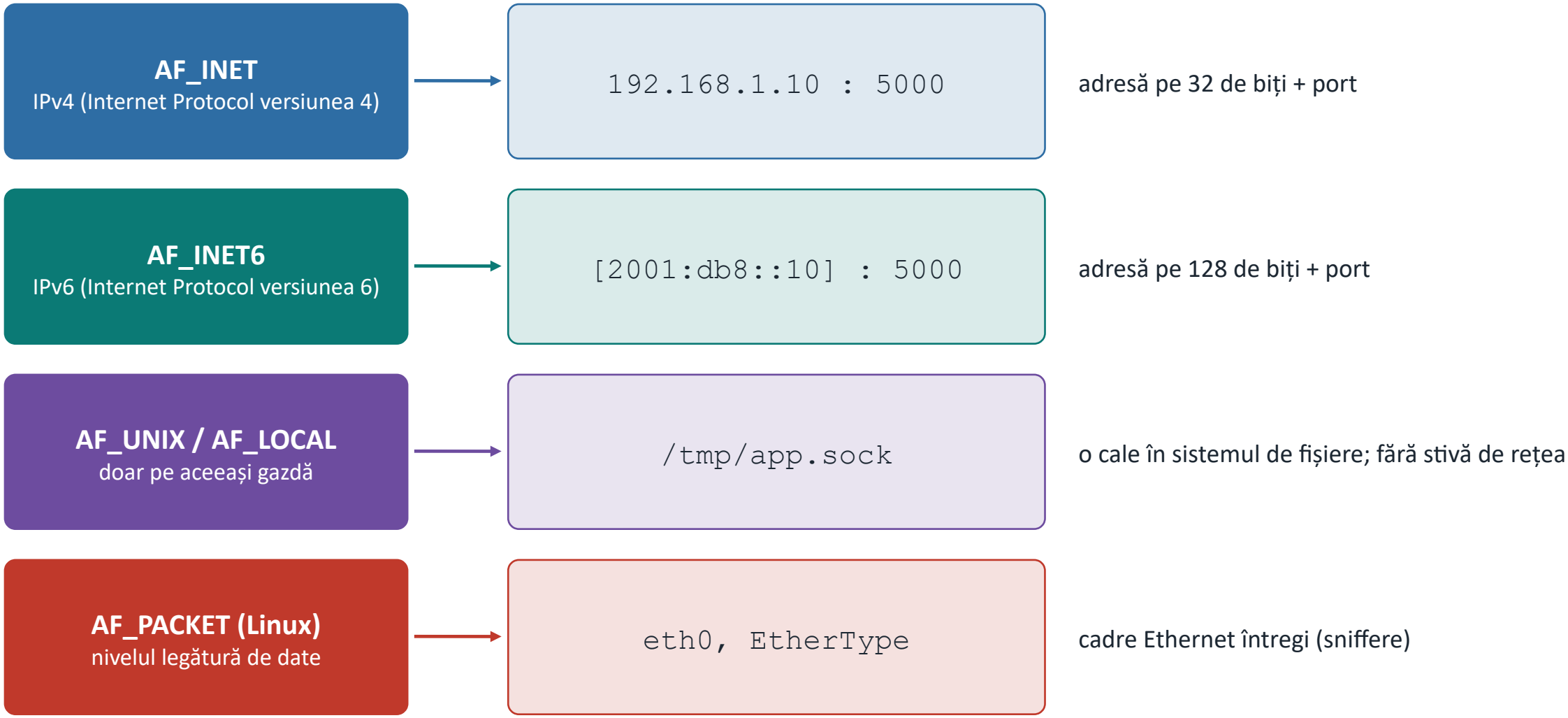
SOCK_RAW (+ SEQPACKET)



programul își construiește singur antetele

- ocolește TCP (Transmission Control Protocol)/UDP (User Datagram Protocol)
- necesită drepturi de admin / root
- ping, traceroute, instrumente de pachete
- nu există în java.net
- **SEQPACKET: mesaje fiabile (SCTP (Stream Control Transmission Protocol))**

După familia de adrese (domeniul de comunicație)



Java: InetAddress ascunde diferența IPv4 (Internet Protocol versiunea 4)/IPv6 (Internet Protocol versiunea 6); UnixDomainSocketAddress (Java 16+) pentru AF_UNIX — și în Windows 10 1803+.

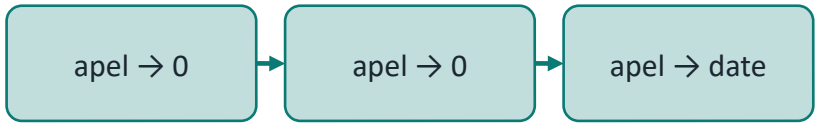
După modelul I/O (Input/Output – intrare/ieșire): ce se întâmplă cât timp programul așteaptă

Blocant (implicit în java.net)



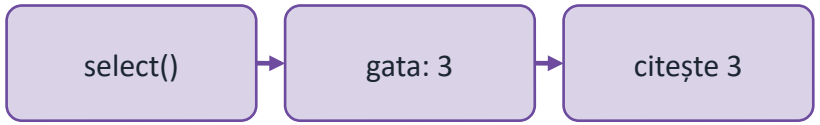
read() suspendă firul până sosesc date
cod simplu; un fir pentru fiecare conexiune

Neblocant



read() revine imediat: date sau „încă nimic”
programul trebuie să interogheze repetat — irosește CPU (Central Processing Unit – procesor) dacă e folosit singur

Multiplexat



un singur fir întreabă: care dintre socket-urile mele sunt gata?
select/poll/epoll, Windows IOCP (Input/Output Completion Ports – porturi de finalizare I/O), Java NIO (New Input/Output – noua bibliotecă de intrare/ieșire) Selector

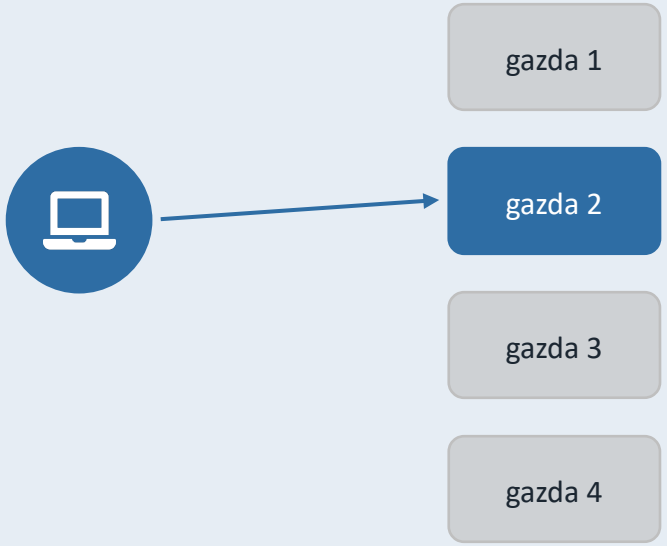
Asincron



pornește citirea; OS (Operating System – sistem de operare) vă anunță când s-a terminat
Java AsynchronousSocketChannel, callback-uri/futures

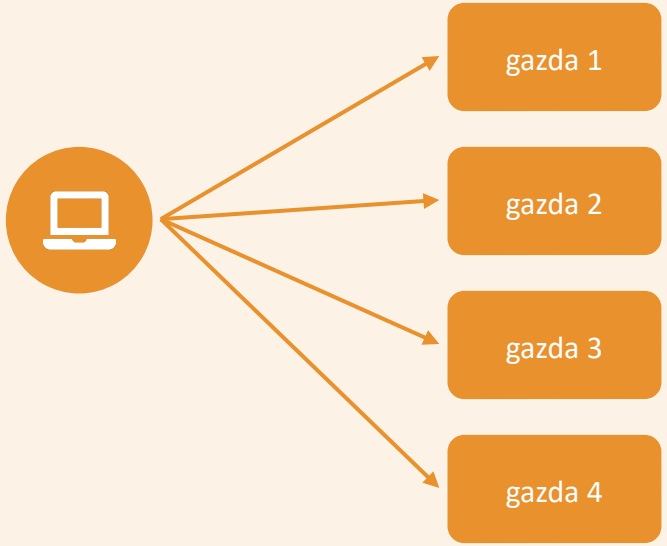
După modul de livrare: unicast, broadcast, multicast

Unicast



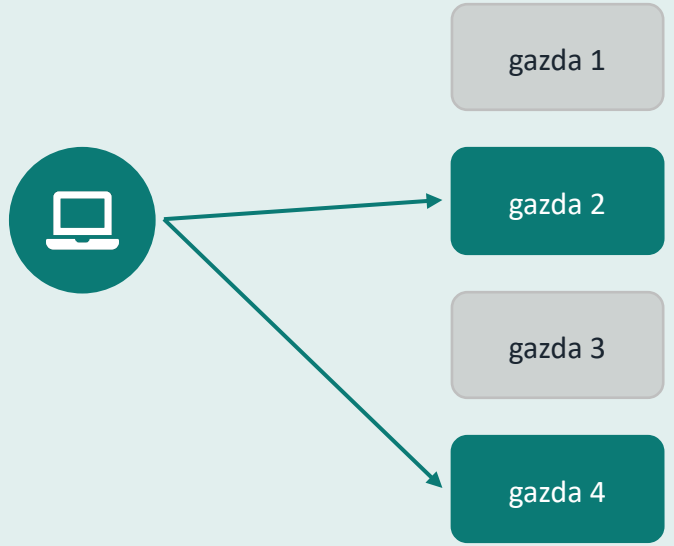
un emițător → un receptor
TCP (Transmission Control Protocol) și UDP (User Datagram Protocol)

Broadcast



un emițător → toate gazdele din subrețea
doar UDP (User Datagram Protocol) — DatagramSocket.setBroadcast(true), de ex. 255.255.255.255 (DHCP (Dynamic Host Configuration Protocol))

Multicast



un emițător → gazdele care s-au înscris într-un grup
doar UDP (User Datagram Protocol) — MulticastSocket.joinGroup(224.x.x.x – 239.x.x.x)

4

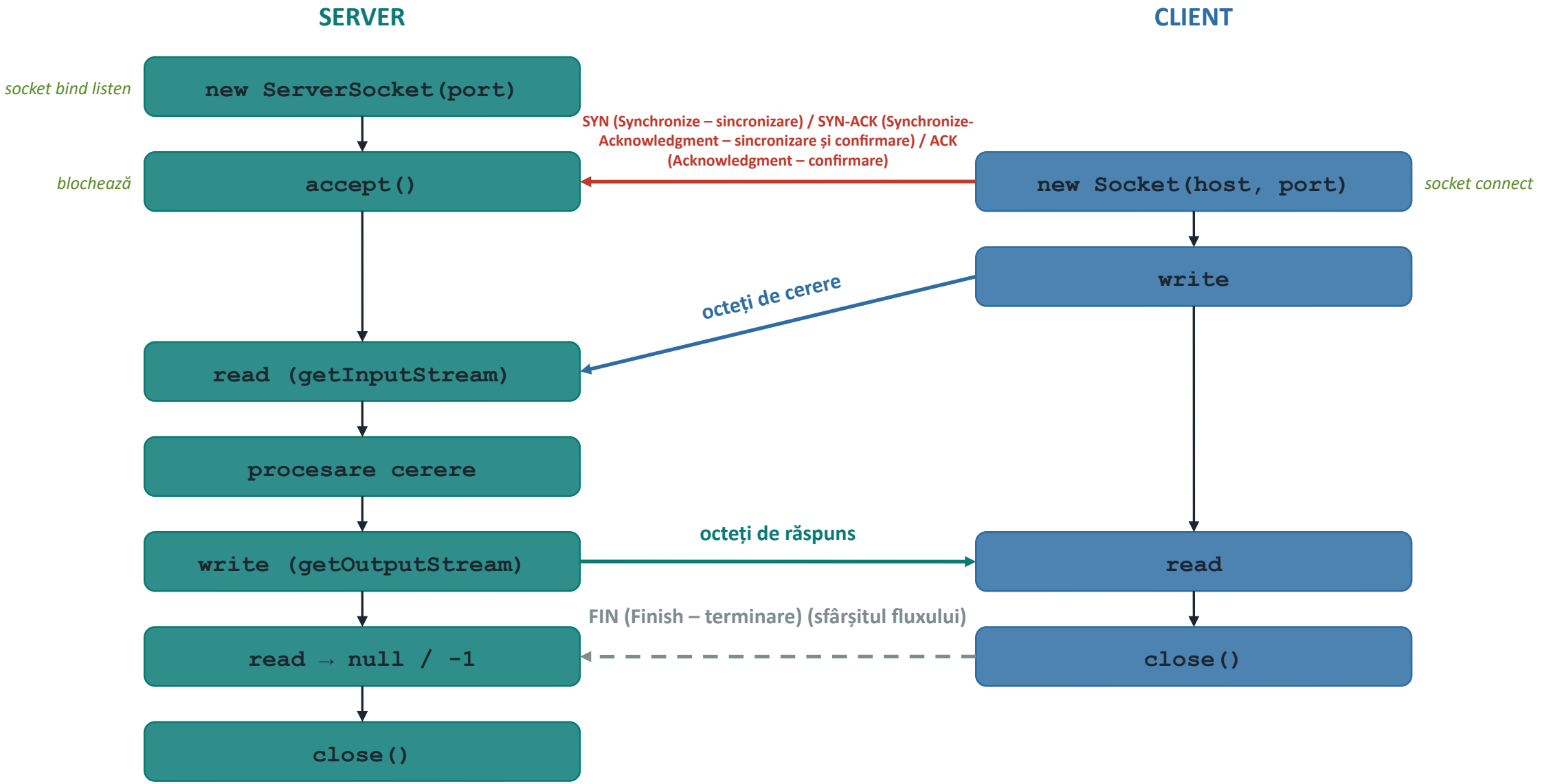
API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket și secvențele de apeluri

Primitivele Berkeley și ce trimite fiecare în rețea

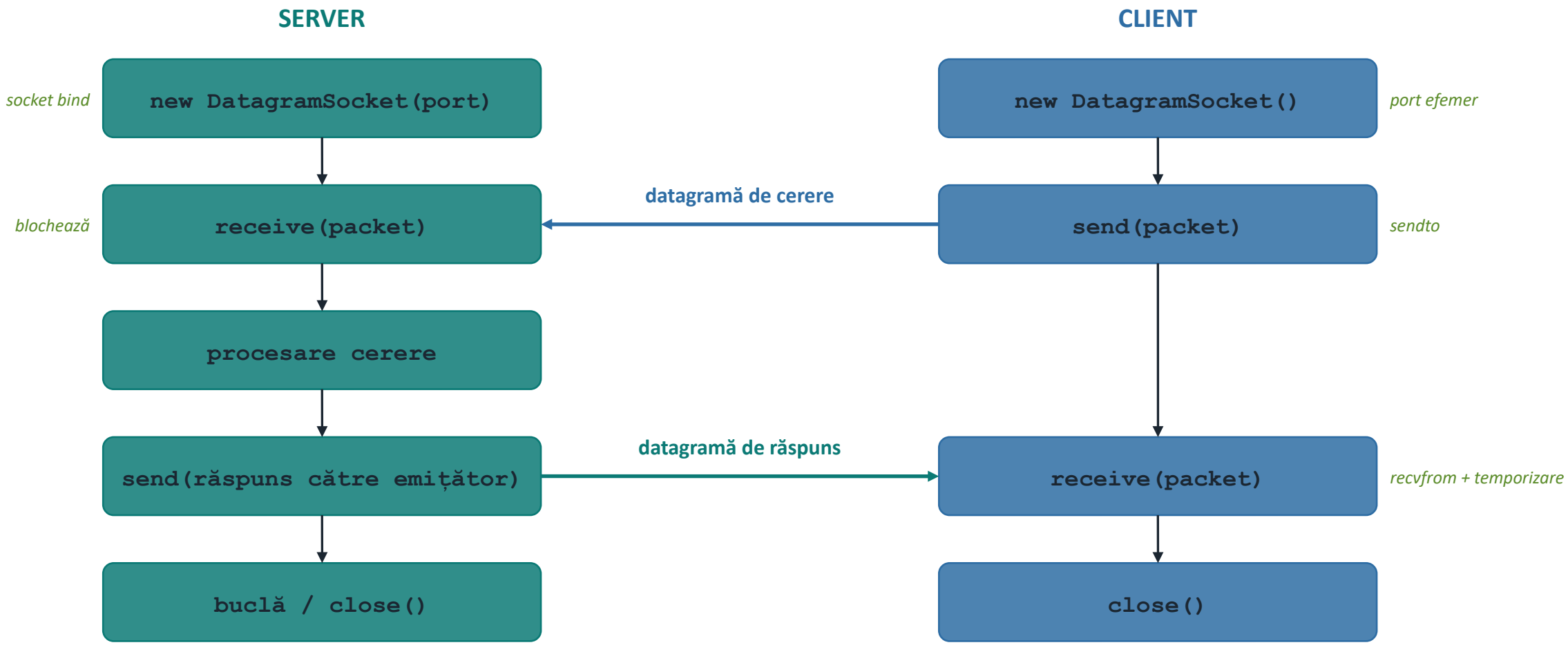
Primitivele Berkeley sockets

Primitivă	Scop	În rețea	Folosită de
`socket()`	creează un socket (familie, tip)	—	ambele
`bind()`	asociază IP (Internet Protocol) local + port	—	servere
`listen()`	mod pasiv + coada backlog	portul în LISTEN	Server TCP (Transmission Control Protocol)
`accept()`	preia o conexiune stabilită → socket nou	(stabilire făcută de nucleu)	Server TCP (Transmission Control Protocol)
`connect()`	deschide conexiunea către adresa la distanță	SYN (Synchronize – sincronizare) → SYN-ACK (Synchronize-Acknowledgment – sincronizare și confirmare) → ACK (Acknowledgment – confirmare)	Client TCP (Transmission Control Protocol)
`send()` / `recv()`	octeți pe un socket conectat	segmente de date + ACKs (Acknowledgments – confirmări)	TCP (Transmission Control Protocol)
`sendto()` / `recvfrom()`	o datagramă către / de la o adresă	o datagramă UDP (User Datagram Protocol)	UDP (User Datagram Protocol)
`shutdown()`	închide un sens	FIN (Finish – terminare)	TCP (Transmission Control Protocol)
`close()`	eliberează socket-ul	FIN (Finish – terminare) (sau RST (Reset – resetare))	ambele

TCP (Transmission Control Protocol): secvența de apeluri (nume Java, apeluri POSIX (Portable Operating System Interface) cu verde)



UDP (User Datagram Protocol): secvența de apeluri — fără conexiune, fără accept

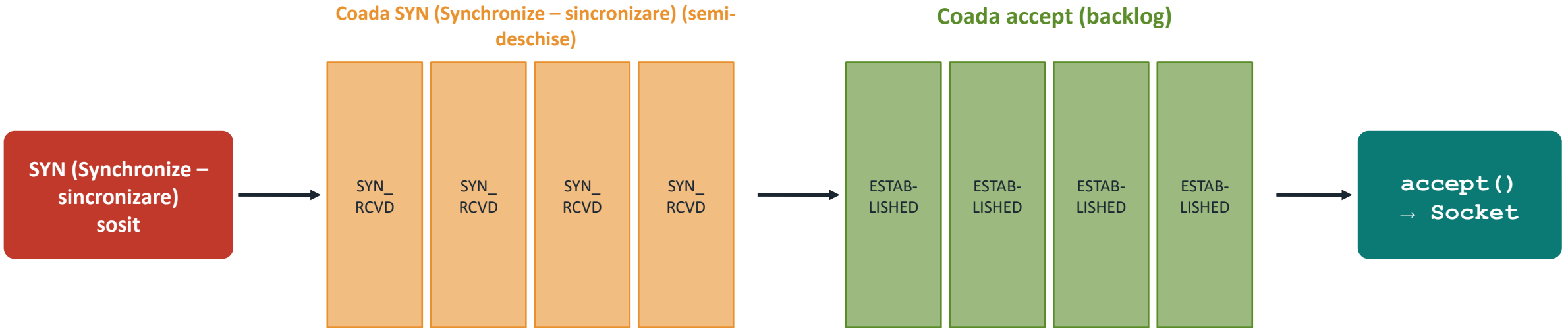


Un singur socket de server pentru toți clienții · adresa de răspuns se ia din pachetul primit

De la apeluri la pachete: o sesiune TCP (Transmission Control Protocol) de ecou



În interiorul listen() și accept(): cele două cozi din nucleu



ACK (Acknowledgment – confirmare)-ul final al stabilirii mută o conexiune din coada SYN (Synchronize – sincronizare) în coada accept

Ce înseamnă

- stabilirea conexiunii este finalizată de NUCLEU, chiar înainte de apelul accept()
- accept() doar preia următoarea conexiune gata
- new ServerSocket(port, backlog) stabilește lungimea cozii accept (implicit 50 în Java)

Când coada este plină

- SYNs (Synchronize – segmente de sincronizare) noi sunt aruncate (Linux) sau refuzate
- clienții văd conectări lente sau eșuate
- → apelați rapid accept(); dați clienții unor fire de execuție

5

Programarea în rețea în Java

Clasele java.net, corespondența cu API (Application Programming Interface – interfață de programare a aplicațiilor)-ul și excepțiile

Harta java.net pentru programarea cu socket-uri

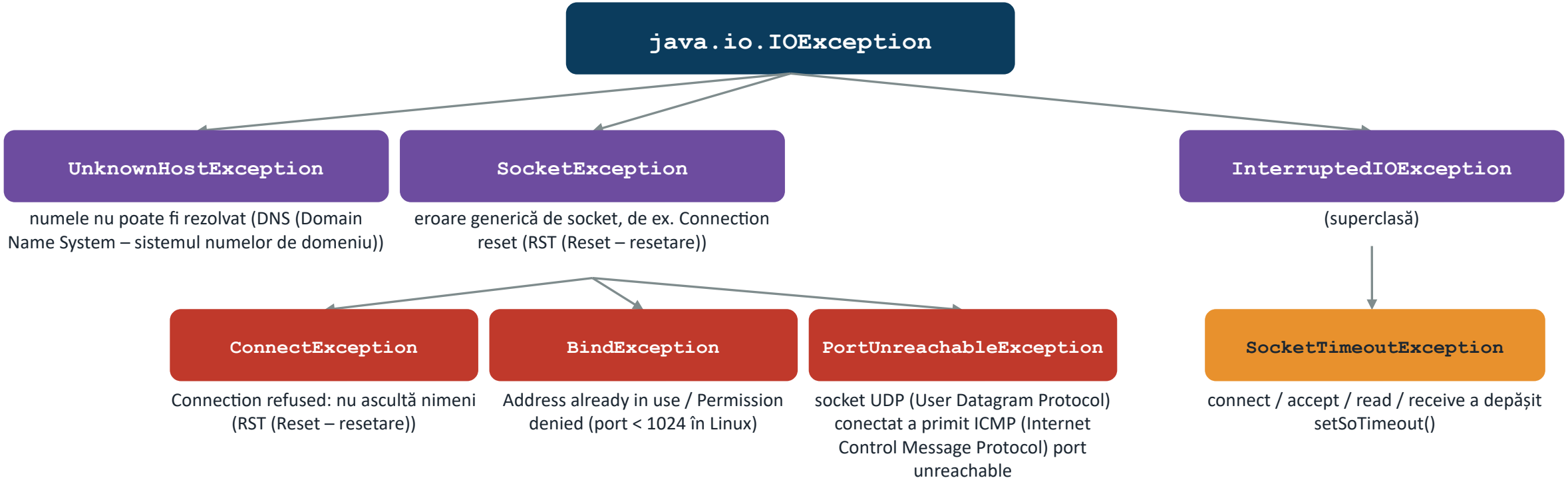
Adrese	TCP (Transmission Control Protocol) (stream)	UDP (User Datagram Protocol) (datagramă)	Dincolo de acest curs
`InetAddress` adresă IP (Internet Protocol) + căutare DNS (Domain Name System – sistemul numelor de domeniu)	**`ServerSocket`** socket de ascultare: bind + listen + accept()	**`DatagramSocket`** send() / receive() de datagrame	**`java.nio.channels`** SocketChannel, Selector (neblocant)
`Inet4Address / Inet6Address` subclase concrete	**`Socket`** socket conectat: fluxuri de intrare/ieșire	**`DatagramPacket`** o datagramă: date + lungime + adresă	**`AsynchronousSocketChannel`** callback-uri / futures
`InetSocketAddress` adresă IP (Internet Protocol) + port		**`MulticastSocket`** joinGroup() / leaveGroup()	**`java.net.http.HttpClient`** un protocol întreg (HTTP (Hypertext Transfer Protocol))

`import java.net.*;` `import java.io.*;` — aici sunt toate clasele folosite la laborator.

De la API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket din C la Java

C / POSIX (Portable Operating System Interface) / Winsock		Java (java.net)
`socket() + bind() + listen()`	→	`new ServerSocket(port, backlog)`
`accept()`	→	`serverSocket.accept() → Socket`
`socket() + connect()`	→	`new Socket(host, port)`
`send() / recv()`	→	`getOutputStream().write() / getInputStream().read()`
`socket() + bind() (UDP (User Datagram Protocol))`	→	`new DatagramSocket(port)`
`sendto()`	→	`datagramSocket.send(packet)`
`recvfrom()`	→	`datagramSocket.receive(packet)`
`setsockopt(SO_RCVTIMEO)`	→	`setSoTimeout(ms)`
`shutdown(SHUT_WR) / close()`	→	`shutdownOutput() / close()`
`gethostbyname() / getaddrinfo()`	→	`InetAddress.getByName(name)`

Excepțiile de rețea: ierarhie și cauze



Bună practică: prindeți excepția specifică, spuneți utilizatorului ce s-a întâmplat (gazdă greșită? server oprit? port ocupat? expirare?) și închideți întotdeauna socket-urile cu try-with-resources, astfel încât o eroare să nu lase în urmă o conexiune semi-deschisă.

InetAddress: nume, adrese și DNS (Domain Name System – sistemul numelor de domeniu)

```
import java.net.*;

public class AddressInfo {
  public static void main(String[] a) throws Exception {
    String name = "www.example.com";
    for (InetAddress x : InetAddress.getAllByName(name)) // DNS (Domain
Name System – sistemul numelor de domeniu)
      System.out.println(name + " -> " + x.getHostAddress());
    System.out.println(InetAddress.getLocalHost());
    System.out.println(InetAddress.getLoopbackAddress());
  }
}
```

rezultat posibil

```
www.example.com -> 93.184.215.14
www.example.com -> 2606:2800:21f:cb07:6820:80da:af6b:8b2c
lab-pc-07/192.168.10.21
localhost/127.0.0.1
```

1. **getByName / getAllByName**
   rezolvă un nume de gazdă (înregistrări A + AAAA); aruncă UnknownHostException

2. **getHostAddress()**
   forma numerică, ca text

3. **getLocalHost()**
   numele și adresa acestui calculator

4. **getLoopbackAddress()**
   127.0.0.1 (sau ::1) — teste pe un singur PC (Personal Computer – calculator personal)

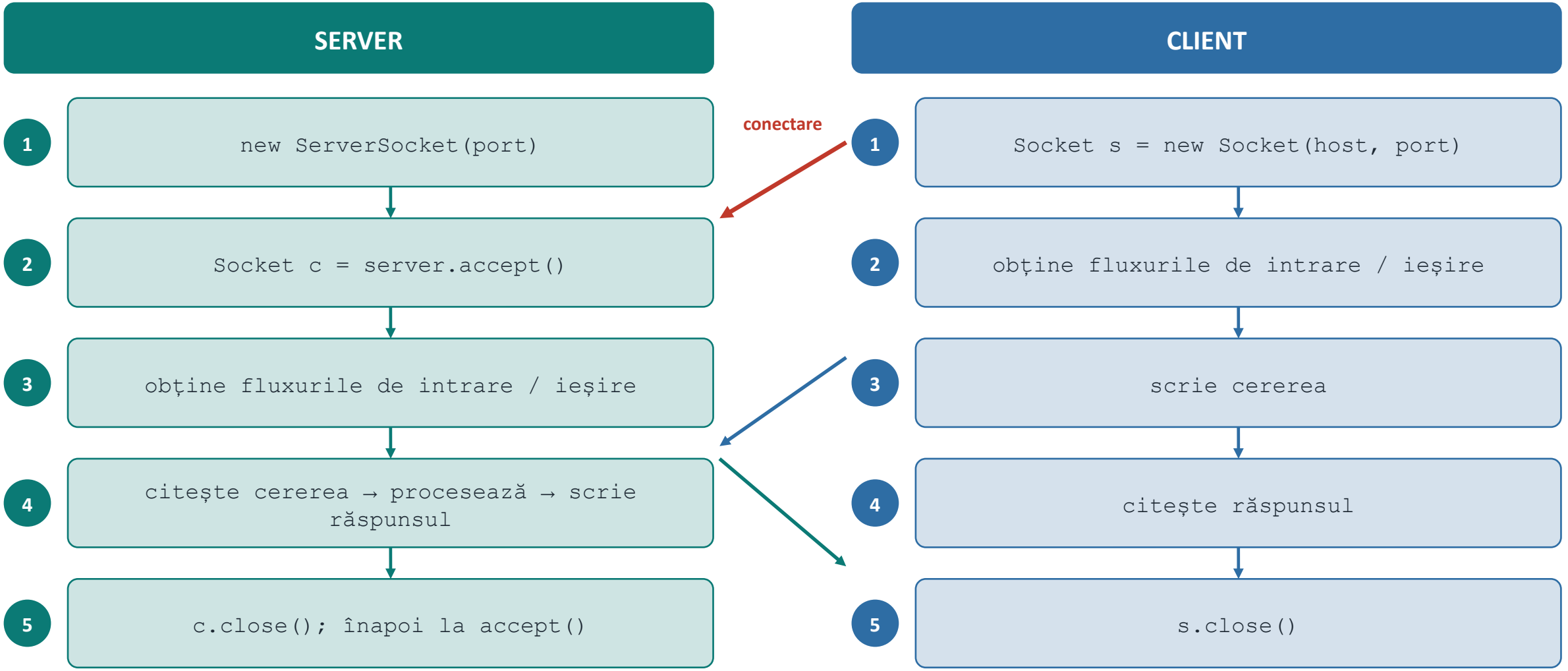
5. **IPv4 (Internet Protocol versiunea 4) sau IPv6 (Internet Protocol versiunea 6)**
   același cod funcționează pentru ambele: Inet4Address / Inet6Address

6

Client-server TCP (Transmission Control Protocol) în Java

ServerSocket, Socket, fluxuri, concurență și delimitarea mesajelor

TCP (Transmission Control Protocol) în Java: pașii pe ambele părți



TcpEchoServer.java — adnotat

```
public class TcpEchoServer {
  public static void main(String[] args) throws IOException {
    int port = args.length > 0 ? Integer.parseInt(args[0]) : 5000;
    try (ServerSocket server = new ServerSocket(port)) {   // (1)
      while (true) {
        Socket client = server.accept();                   // (2)
        new Thread(() -> handle(client)).start();          // (3)
      }
    }
  }
  static void handle(Socket s) {
    try (s;
         BufferedReader in = new BufferedReader(
           new InputStreamReader(s.getInputStream(), "UTF-8"));   // (4)
         PrintWriter out = new PrintWriter(
           new OutputStreamWriter(s.getOutputStream(), "UTF-8"), true)) {
      String line;
      while ((line = in.readLine()) != null) {             // (5)
        if (line.equalsIgnoreCase("BYE")) { out.println("Goodbye"); break; }
        out.println("ECHO: " + line.toUpperCase());        // (6)
      }
    } catch (IOException e) { System.out.println("Error: " + e.getMessage()); }
  }                                                        // (7) auto-close
}
```

1. **ServerSocket(port)**
   socket + bind + listen; portul este acum LISTEN

2. **accept()**
   blochează; întoarce un Socket conectat NOU

3. **un fir pentru fiecare client**
   bucla revine imediat la accept()

4. **fluxuri + UTF-8 (8-bit Unicode Transformation Format – format Unicode pe 8 biți)**
   octeți ↔ text, la fel în Windows/Linux

5. **readLine()**
   un mesaj = o linie; null = celălalt capăt a trimis FIN (Finish – terminare)

6. **println + auto-flush**
   răspunsul este împins imediat către TCP (Transmission Control Protocol)

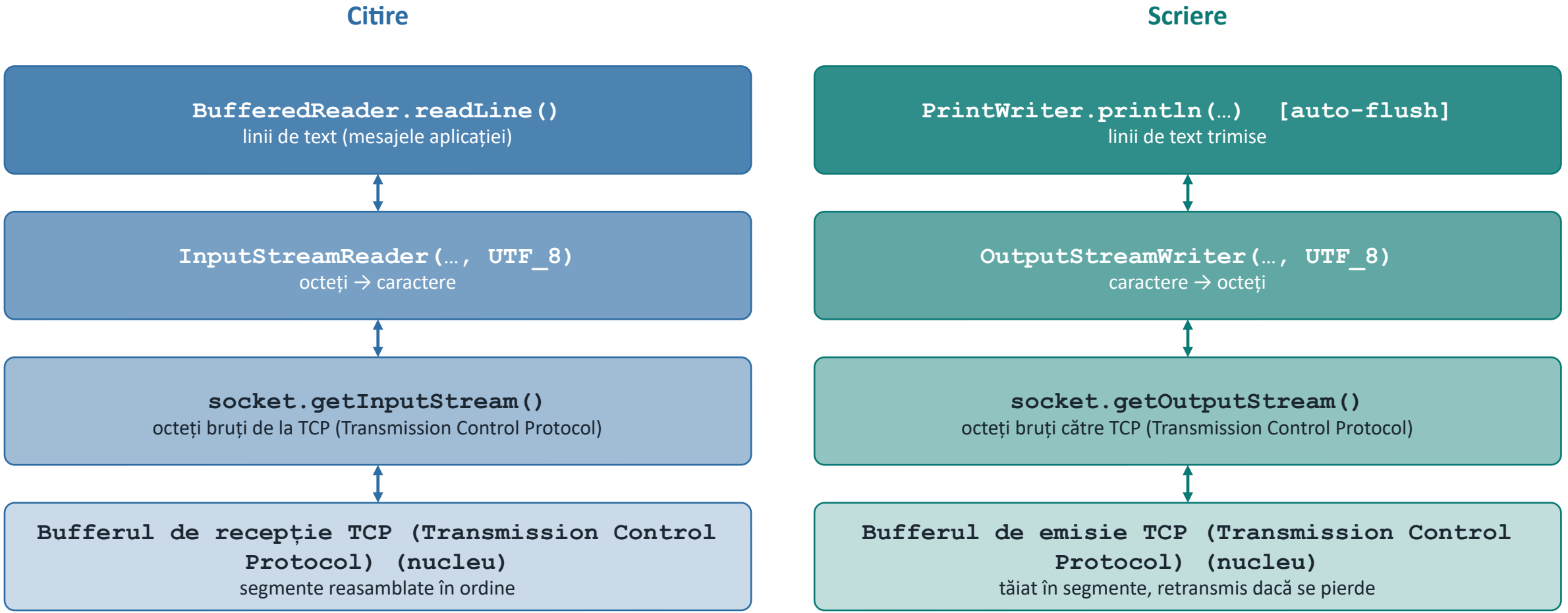
7. **try-with-resources**
   închide fluxurile și socket-ul → FIN (Finish – terminare)

TcpEchoClient.java — adnotat

```
public class TcpEchoClient {
  public static void main(String[] args) throws IOException {
    String host = args.length > 0 ? args[0] : "localhost";
    int port = args.length > 1 ? Integer.parseInt(args[1]) : 5000;
    try (Socket s = new Socket(host, port);                    // (1)
         BufferedReader in = new BufferedReader(
           new InputStreamReader(s.getInputStream(), "UTF-8"));
         PrintWriter out = new PrintWriter(
           new OutputStreamWriter(s.getOutputStream(), "UTF-8"), true);
         BufferedReader kb = new BufferedReader(
           new InputStreamReader(System.in))) {                // (2)
      System.out.println("Local port: " + s.getLocalPort());  // (3)
      String line;
      while ((line = kb.readLine()) != null) {
        out.println(line);                                     // (4)
        String reply = in.readLine();                          // (5)
        if (reply == null) break;
        System.out.println(reply);
        if (line.equalsIgnoreCase("BYE")) break;               // (6)
      }
    }                                                          // (7)
  }
}
```

1. **new Socket(host, port)**
   DNS (Domain Name System – sistemul numelor de domeniu) + port efemer + stabilirea conexiunii în trei pași
2. **cititor de la tastatură**
   liniile tastate de utilizator
3. **getLocalPort()**
   portul efemer ales de OS (Operating System – sistem de operare)
4. **println(line)**
   cerere = o linie, golită imediat
5. **readLine()**
   așteaptă linia de răspuns a serverului
6. **BYE**
   sfârșitul sesiunii la nivelul aplicației
7. **închidere automată**
   close() trimite FIN (Finish – terminare)

Straturile de fluxuri pe un socket TCP (Transmission Control Protocol)



Alternative: DataInputStream / DataOutputStream pentru date binare (readInt, writeInt, readFully).

Server iterativ comparativ cu server concurent: cine așteaptă?

Server iterativ: câte un client pe rând



Server concurent: un fir pentru fiecare client



Aceeași muncă, dar serverul concurent termină toți trei clienții la t = 4 în loc de t = 10.

Modele de concurență pentru servere

Un fir pentru fiecare client

```
accept → new Thread(handle).start()
```

simplu; dar mii de clienți = mii de fire ale OS (Operating System – sistem de operare)

Bazin de fire (thread pool)

```
accept → pool.submit(() -> handle(c))
Executors.newFixedThreadPool(50)
```

resurse limitate; dimensiunea bazinului limitează concurența

Fire virtuale (Java 21+)

```
Executors.newVirtualThreadPerTaskExecutor()
```

stil de cod blocant care scalează la foarte mulți clienți

Bazat pe evenimente (NIO (New Input/Output – noua bibliotecă de intrare/ieșire))

```
un singur fir + Selector peste canale neblocante
```

scalabilitate maximă; cod mai complex

Delimitarea mesajelor: TCP (Transmission Control Protocol) nu are limite ale mesajelor

Emițătorul: write("HELLO") apoi write("WORLD") → read(buf) al receptorului poate întoarce…

`HELLOWORLD`	`HELLO | WORLD`	`HEL | LOWORLD`
ambele într-o citire	*cum au fost scrise (noroc)*	*tăiat oriunde*

Aplicația trebuie să definească unde se termină un mesaj — trei soluții:

Delimitator	`HELLO\n WORLD\n`	readLine(); folosit de antetele HTTP (Hypertext Transfer Protocol), SMTP (Simple Mail Transfer Protocol), protocolul nostru de ecou
Prefix de lungime	`[00 00 00 05] HELLO [00 00 00 05] WORLD`	writeInt(len) + octeți; readInt() + readFully(); protocoale binare
Dimensiune fixă	`fiecare mesaj are exact N octeți`	readFully(buf, 0, N); simplu, dar rigid

Delimitarea cu prefix de lungime în Java



writeInt(5): 4 octeți, big-endian

5 octeți de date UTF-8 (8-bit Unicode Transformation Format – format Unicode pe 8 biți)

```
// emițător
DataOutputStream out = new DataOutputStream(
    new BufferedOutputStream(sock.getOutputStream()));
byte[] m = text.getBytes(StandardCharsets.UTF_8);
out.writeInt(m.length);   out.write(m);   out.flush();
```

```
// receptor
DataInputStream in = new DataInputStream(
    new BufferedInputStream(sock.getInputStream()));
int len = in.readInt();
byte[] b = new byte[len];  in.readFully(b);
```

De ce readFully()?

- read(buf) poate întoarce MAI PUȚINI octeți decât s-au cerut (cât a sosit); readFully repetă până când în buffer sunt exact len octeți
- și readInt() așteaptă toți cei 4 octeți → limita mesajului este găsită întotdeauna

Opțiuni utile ale socket-urilor TCP (Transmission Control Protocol)

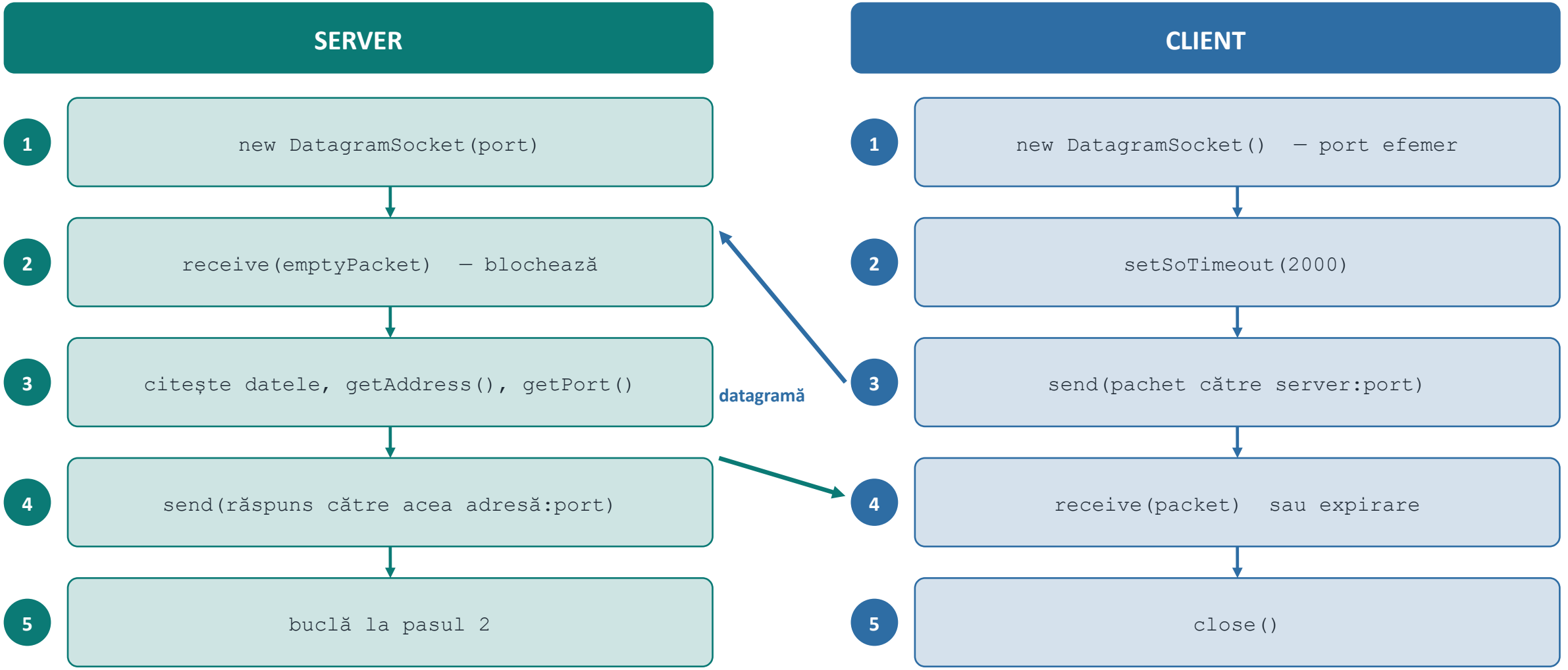
`setSoTimeout(ms)`	timpul maxim de blocare pentru read / accept → *SocketTimeoutException*
`connect(addr, ms)`	limitează timpul de stabilire a conexiunii *new Socket() apoi connect()*
`setTcpNoDelay(true)`	dezactivează Nagle: scrierile mici pleacă imediat *aplicații interactive, jocuri*
`setKeepAlive(true)`	sonde pe conexiunile inactive *detectează capetele căzute*
`setReuseAddress(true)`	reasociază un port cu resturi în TIME_WAIT *repornire rapidă a serverelor*
`set…BufferSize(n)`	dimensiunea bufferului din nucleu *influențează rwnd anunțată*
`shutdownOutput()`	semi-închidere: trimite FIN (Finish – terminare), continuă să citească *„Am terminat, dă-mi răspunsul”*
`ServerSocket(p, backlog, addr)`	lungimea cozii; asociere cu o singură interfață *127.0.0.1 = doar local*

7

Client-server UDP (User Datagram Protocol) în Java

DatagramSocket, DatagramPacket și fiabilitate construită manual

UDP (User Datagram Protocol) în Java: pașii pe ambele părți



Fără ServerSocket, fără accept(), fără fluxuri — doar pachete.

UdpEchoServer.java — adnotat

```
public class UdpEchoServer {
  public static void main(String[] args) throws Exception {
    int port = args.length > 0 ? Integer.parseInt(args[0]) : 6000;
    try (DatagramSocket socket = new DatagramSocket(port)) {    // (1)
      byte[] buf = new byte[1024];
      while (true) {
        DatagramPacket req = new DatagramPacket(buf, buf.length); // (2)
        socket.receive(req);                                      // (3)
        String msg = new String(req.getData(), 0, req.getLength(),
                                StandardCharsets.UTF_8);          // (4)
        byte[] ans = ("ECHO: " + msg.toUpperCase())
                       .getBytes(StandardCharsets.UTF_8);
        socket.send(new DatagramPacket(ans, ans.length,
                    req.getAddress(), req.getPort()));            // (5)
      }
    }
  }
}
```

1. **DatagramSocket(port)**
   socket + bind pe un port cunoscut

2. **pachet gol**
   un buffer de 1024 de octeți pentru recepție

3. **receive()**
   blochează; completează bufferul, lungimea și adresa emițătorului

4. **getLength()!**
   doar octeții ACESTEI datagrame

5. **răspuns către emițător**
   adresa și portul sunt luate din cerere

Un singur socket servește toți clienții — fără stare per client, fără fire de execuție pentru cereri scurte

UdpEchoClient.java — adnotat

```
public class UdpEchoClient {
  public static void main(String[] args) throws Exception {
    InetAddress server = InetAddress.getByName(
        args.length > 0 ? args[0] : "localhost");              // (1)
    int port = args.length > 1 ? Integer.parseInt(args[1]) : 6000;
    try (DatagramSocket socket = new DatagramSocket();          // (2)
         BufferedReader kb = new BufferedReader(
           new InputStreamReader(System.in))) {
      socket.setSoTimeout(2000);                                // (3)
      String line;
      while ((line = kb.readLine()) != null && !line.equalsIgnoreCase("BYE")) {
        byte[] data = line.getBytes(StandardCharsets.UTF_8);
        socket.send(new DatagramPacket(data, data.length,
                                       server, port));          // (4)
        DatagramPacket resp = new DatagramPacket(new byte[1024], 1024);
        try {
          socket.receive(resp);                                 // (5)
          System.out.println(new String(resp.getData(), 0,
                     resp.getLength(), StandardCharsets.UTF_8));
        } catch (SocketTimeoutException e) {                    // (6)
          System.out.println("Niciun raspuns in 2 s");
        }
      }
    }
  }
}
```

1. **rezolvă serverul**
   InetAddress pentru destinație

2. **DatagramSocket()**
   fără port dat → port efemer

3. **setSoTimeout**
   UDP (User Datagram Protocol) nu oferă garanții: nu așteptați la nesfârșit

4. **send(packet)**
   date + adresă și port destinație

5. **receive(resp)**
   datagrama de răspuns (pachet nou)

6. **expirare**
   cererea sau răspunsul s-au pierdut, ori serverul e oprit

Anatomia unui DatagramPacket

Un obiect pachet = buffer + deplasament + lungime + adresă la distanță + port la distanță



buffer de 1024 de octeți (sunt desenați doar 16)

getLength() = 5

octeți vechi dintr-un mesaj anterior, mai lung

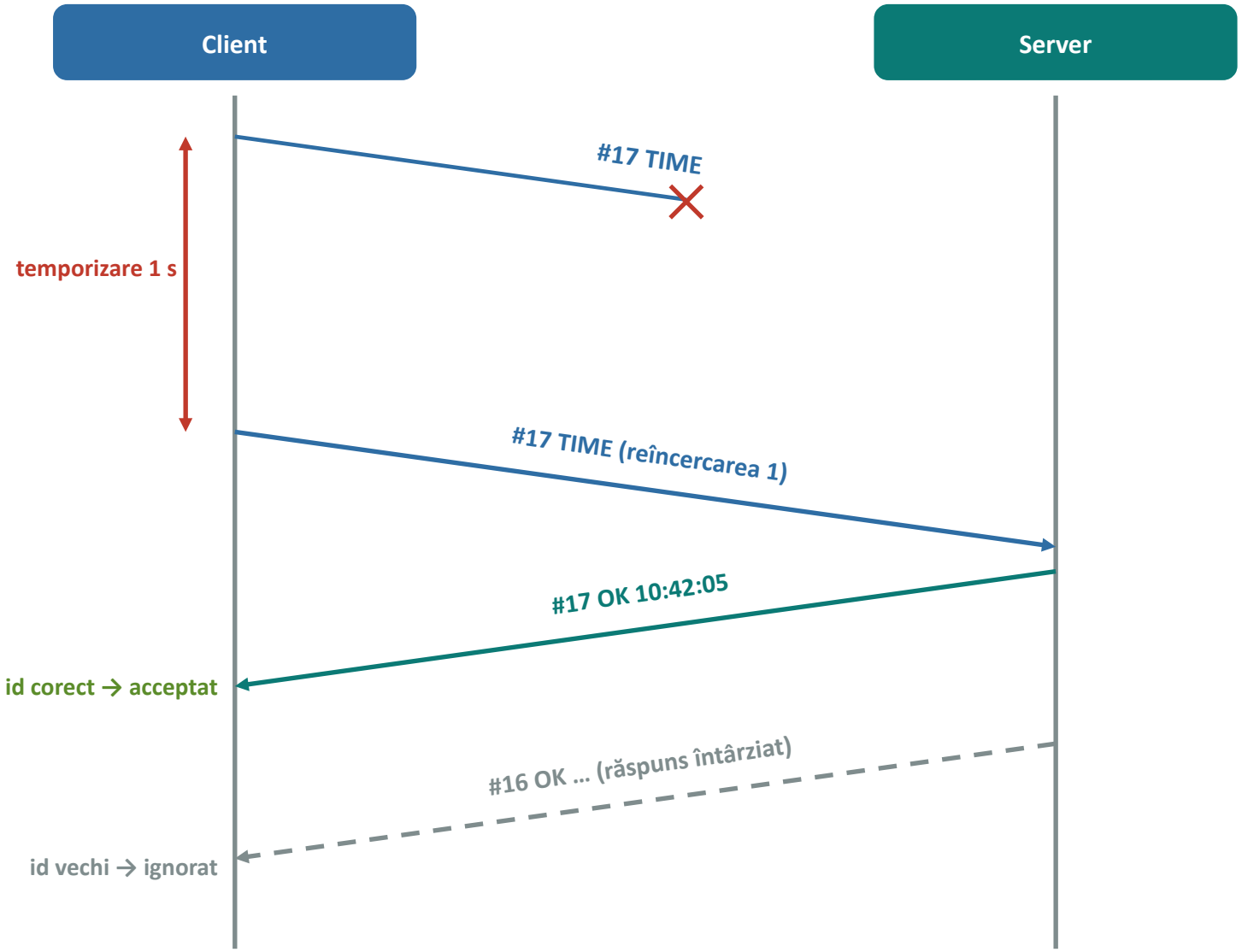
Recepție

- new DatagramPacket(buf, buf.length)
- după receive(): getData() = tot bufferul, getLength() = dimensiunea acestei datagrame
- getAddress() / getPort() = EMIȚĂTORUL
- receive() micșorează lungimea pachetului → refolosirea necesită setLength(buf.length)
- datagramă mai mare decât bufferul → octeții în plus se pierd fără avertisment

Emisie

- new DatagramPacket(data, data.length, address, port)
- adresă + port = DESTINAȚIA
- un send() = exact o datagramă UDP (User Datagram Protocol)
- decodificare corectă:
- new String(p.getData(), 0, p.getLength(), UTF_8)

UDP (User Datagram Protocol) suficient de fiabil: temporizare, reîncercare, identificator de cerere



Rețetă (tema A2)

- puneți un identificator de cerere în fiecare mesaj
- așteptați cu setSoTimeout()
- la expirare: retrimiteți, de cel mult N ori (de ex. 3)
- ignorați răspunsurile cu alt identificator
- preferați cererile idempotente (TIME, ADD 3 4): execuția de două ori nu face rău
- este un mic ARQ (Automatic Repeat reQuest – cerere automată de repetare) stop-and-wait — săptămâna 6!

Cât de mare poate fi o datagramă?

Pachet IPv4 (Internet Protocol versiunea 4) maxim: 65.535 de octeți

Încărcătură utilă UDP (User Datagram Protocol) maximă: 65.535 − 20 (IP (Internet Protocol)) − 8 (UDP (User Datagram Protocol)) = 65.507 octeți

1500

MTU (Maximum Transmission Unit – unitatea maximă de transmisie) Ethernet: 1500 de octeți

1472

încărcătură UDP (User Datagram Protocol) sigură pe Ethernet: 1500 − 20 − 8 = 1472 de octeți

Mai mare decât MTU (Maximum Transmission Unit – unitatea maximă de transmisie)?

- IP (Internet Protocol) fragmentează datagrama
- pierderea UNUI fragment duce la pierderea ÎNTREGII datagrame

Regulă practică

- păstrați datagramele ≤ ~1200–1400 de octeți
- aveți nevoie de mai mult? împărțiți singuri sau folosiți TCP (Transmission Control Protocol)

8

Programarea TCP (Transmission Control Protocol) comparativ cu UDP (User Datagram Protocol), în practică

Comparație, comportament la erori, depanare, testare

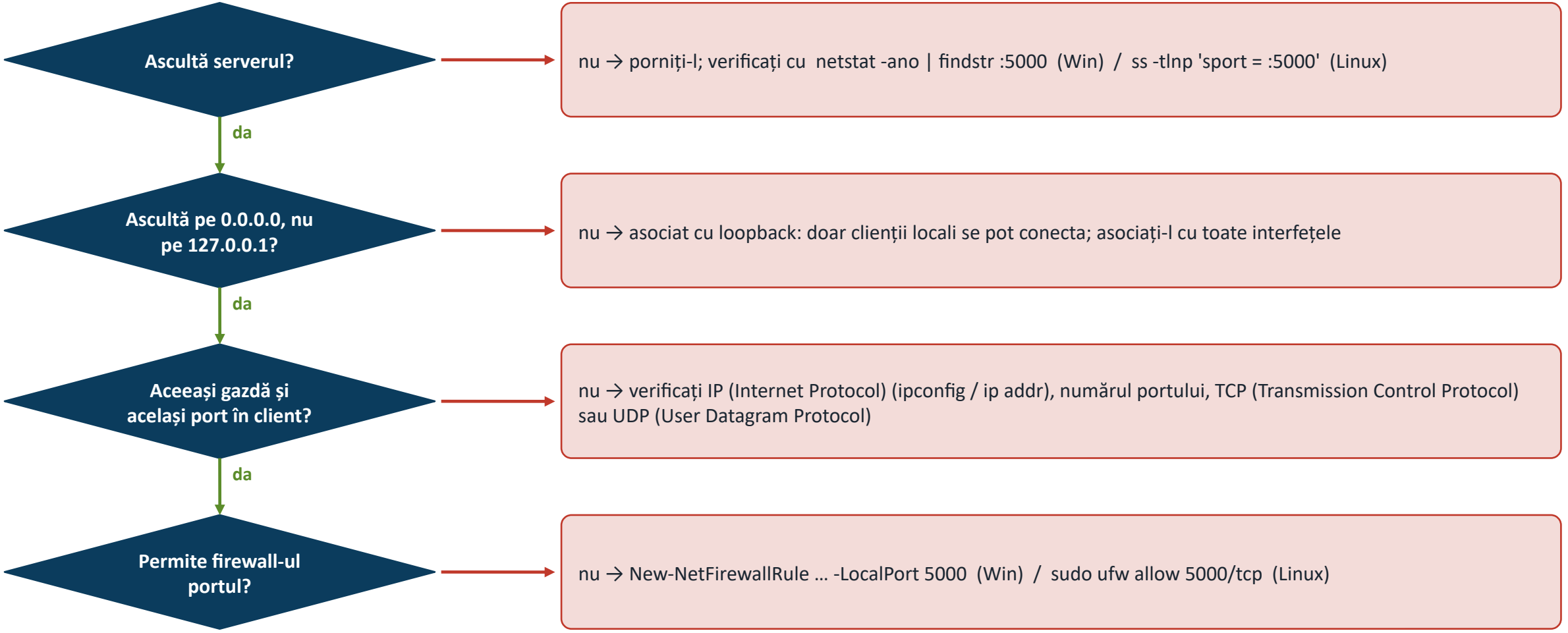
Programarea TCP (Transmission Control Protocol) comparativ cu UDP (User Datagram Protocol)

	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Clase Java	ServerSocket, Socket	DatagramSocket, DatagramPacket
Stabilirea conexiunii	da: new Socket() / accept()	nu există
Socket-uri pe server	1 de ascultare + 1 per client	1 pentru toți clienții
Unitatea de date	flux de octeți → necesită delimitare	datagramă → limitele sunt păstrate
Fiabilitate, ordine	garantate de TCP (Transmission Control Protocol)	sarcina aplicației
Stil I/O (Input/Output – intrare/ieșire)	InputStream / OutputStream	send() / receive() de pachete
Sfârșitul comunicației	read() → -1 / readLine() → null	nevizibil: temporizări sau un mesaj BYE
Broadcast / multicast	imposibil	posibil
Server tipic	concurent (fire de execuție)	o singură buclă

Ce vede programul când ceva nu merge?

	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Serverul nu rulează	ConnectException: Connection refused (RST (Reset – resetare))	send() OK; receive() expiră (socket conectat: PortUnreachableException)
Gazdă oprită / firewall aruncă pachetele	connect() așteaptă, apoi expiră	receive() expiră
Port deja în uz	BindException: Address already in use	BindException: Address already in use
Port < 1024 în Linux (fără root)	BindException: Permission denied	BindException: Permission denied
Server oprit forțat în timpul sesiunii	readLine() → null (FIN (Finish – terminare)) or Connection reset (RST (Reset – resetare))	receive() expiră
Nume de gazdă greșit	UnknownHostException	UnknownHostException

Schemă de depanare: „clientul meu nu se poate conecta”



toate da → capturați cu Wireshark / tcpdump: SYN (Synchronize – sincronizare) fără răspuns = filtrat; RST (Reset – resetare) = port închis; SYN-ACK (Synchronize-Acknowledgment – sincronizare și confirmare) = rețeaua funcționează

Trusa de testare: Windows și Linux

	Windows (PowerShell)	Linux (bash)
Compilare / rulare	`javac *.java` · `java TcpEchoServer 5000`	`javac *.java` · `java TcpEchoServer 5000`
Cine ascultă?	`netstat -ano | findstr :5000` `Get-NetTCPConnection -LocalPort 5000`	`ss -tlnp 'sport = :5000'` `ss -uanp 'sport = :6000'`
Client generic	`ncat localhost 5000` · `ncat -u … 6000` `Test-NetConnection localhost -Port 5000`	`nc localhost 5000` · `nc -u … 6000` `nc -zv host 5000`
Deschiderea firewall-ului	`New-NetFirewallRule -DisplayName "CN port 5000" -Direction Inbound -Protocol TCP -LocalPort 5000 -Action Allow`	`sudo ufw allow 5000/tcp` `sudo firewall-cmd --add-port=5000/tcp`
Urmărirea pachetelor	`Wireshark (adaptorul loopback Npcap)` `filtru: tcp.port == 5000`	`sudo tcpdump -i lo -nn -A 'tcp port 5000'`
Firele de execuție ale serverului	`jcmd` · `jstack <pid>`	`jcmd` · `jstack <pid>` · `ps -o nlwp= -p <pid>`

Rezumat: comunicația prin socket-uri pe o singură pagină

